

PCA e HPC: Otimização Arquitetural e Paralelismo na Redução de Dimensionalidade

Vinícius César

15 de março de 2026

Resumo

Este trabalho apresenta a implementação e otimização do algoritmo de Análise de Componentes Principais (PCA) em C, com foco em Computação de Alto Desempenho (HPC). O objetivo é reproduzir os resultados da biblioteca Scikit-Learn (Python), superando seu tempo de execução ao explorar a localidade de cache e o paralelismo via OpenMP. O estudo contrasta métodos de resolução implícitos e explícitos, evidenciando o impacto da hierarquia de memória na execução de algoritmos de álgebra linear densa.

1 Introdução

A Análise de Componentes Principais (PCA) é uma das técnicas mais fundamentais no ecossistema de Aprendizado de Máquina para redução de dimensionalidade e extração de características. Atualmente, a resolução desse problema é amplamente dominada por bibliotecas em Python, como o Scikit-Learn e o NumPy. Embora o Python atue como uma interface de alto nível, a carga computacional real é delegada a bibliotecas de baixo nível (como BLAS e LAPACK), otimizadas para arquiteturas modernas.

Apesar da facilidade de uso dessas ferramentas, a abstração oculta gargalos arquiteturais críticos. O objetivo deste trabalho é desenvolver uma solução nativa em C, a partir do zero, capaz de obter exatamente os mesmos resultados numéricos do Scikit-Learn para o PCA, porém buscando um desempenho superior ou equivalente por meio da aplicação rigorosa de técnicas de Computação de Alto Desempenho (HPC).

2 Fundamentação Teórica

2.1 O Problema do PCA e os Multiplicadores de Lagrange

O objetivo do PCA é encontrar uma direção (vetor peso w) que maximize a variância dos dados projetados. Dada uma matriz de dados X (centrada na média) de dimensões $m \times n$, a matriz de covariância é definida como $C = \frac{1}{m-1}X^T X$. A variância dos dados projetados em w é dada por $w^T C w$.

Para garantir que o vetor de projeção seja apenas direcional, impomos a restrição de que ele seja um vetor unitário: $w^T w = 1$. O problema de otimização restrita é formulado

utilizando os Multiplicadores de Lagrange (λ):

$$\mathcal{L}(w, \lambda) = w^T C w - \lambda(w^T w - 1) \quad (1)$$

Para encontrar o ponto de máximo, derivamos o Lagrangiano em relação a w e igualamos a zero:

$$\frac{\partial \mathcal{L}}{\partial w} = 2Cw - 2\lambda w = 0 \quad (2)$$

$$Cw = \lambda w \quad (3)$$

A Equação 3 revela a essência do PCA: o problema de maximização da variância reduz-se à solução do clássico problema de autovalores e autovetores da matriz de covariância C . O autovetor w correspondente ao maior autovalor λ representa o Componente Principal primário.

2.2 Conceitos de HPC e Gestão de Memória

Na execução de algoritmos numéricos, a matemática teórica esbarra nos limites físicos do *hardware*. Dois gargalos são fundamentais:

- **Memory Bound vs Compute Bound:** Algoritmos *Memory Bound* são limitados pela lentidão do barramento da memória RAM. Algoritmos *Compute Bound* são limitados apenas pela velocidade da Unidade de Ponto Flutuante (FPU) do processador.
- **Data Locality e Cache:** A CPU busca dados na RAM em blocos. Acessar dados contíguos na memória (localidade espacial) ativa o *Hardware Prefetcher*, armazenando os dados nos rápidos Caches L1/L2/L3 e evitando o custo letal dos *Cache Misses*.

2.3 Métodos Implícitos vs. Explícitos

Neste projeto, duas arquiteturas matemáticas foram testadas para encontrar os autovetores:

1. **Método Implícito (SVD Unilateral):** Aplica Rotações de Givens diretamente na matriz X ($m \times n$). Possui alta precisão, mas exige a leitura contínua de matrizes gigantes da RAM, caracterizando um cenário *Memory Bound*.
2. **Método Explícito (Eigendecomposition da Covariância):** Calcula explicitamente $C = X^T X$, reduzindo o problema a uma matriz quadrada $n \times n$ (500×500). Como essa matriz cabe inteiramente no Cache L3 da CPU, o cálculo dos autovetores torna-se estritamente *Compute Bound*.

3 Metodologia

O ambiente de testes utilizou uma matriz de dados sintéticos X de dimensões 100.000×500 . Para garantir uma comparação justa e validável com a versão *baseline* em Python, os dados foram gerados em C utilizando a mesma semente algorítmica e gerador de números pseudoaleatórios.

A compilação do código em C utilizou o GCC com diretivas agressivas de otimização de *hardware*:

```
gcc -O3 -march=native -funroll-loops -fexpensive-optimizations
      -ffast-math -fopenmp
```

A avaliação de desempenho consistiu em medir o tempo de parede (*wall time*) estrito de cálculo, isolado da alocação inicial, utilizando a diretiva `omp_get_wtime()`. O método paralelo utilizou a API OpenMP. Os autovalores e autovetores resultantes foram exportados em arquivos de texto para validação de paridade com o Scikit-Learn.

4 Resultados e Discussão

Os testes demonstraram uma evolução arquitetural expressiva, com os resultados de tempo sumarizados na Tabela 1.

Tabela 1: Tempos de execução para a matriz de 100.000×500

Método Algorítmico e Arquitetura	Tempo (s)	Status de Gargalo
Implícito Ingênuo (Ponteiros Duplos)	> 5400.00	RAM Thrashing (Timeout)
Implícito Otimizado (Array 1D contíguo)	696.62	Memory Bound
Explícito Ingênuo (Ponteiros Duplos)	957.66	Sub-utilização de Cache
Explícito Otimizado (Serial)	12.73	Compute Bound
Explícito Otimizado (OpenMP - 4 Threads)	3.90	Compute Bound (Pico)
Baseline Python (Scikit-Learn)	~ 12.00	BLAS / MKL Oculto

A transição de um arranjo fragmentado para um arranjo 1D (*Column-Major*) demonstrou a importância da localidade de dados, viabilizando o cálculo Implícito em 696.62s. Contudo, foi a alteração matemática para o Método Explícito que demoliu a barreira do *Memory Bound*.

O cálculo serial de 12.73s igualou o desempenho do ecossistema Scikit-Learn. Ao aplicar paralelismo via OpenMP no cálculo da covariância ($X^T X$), o tempo caiu para cravados 3.90 segundos. A análise de escalabilidade multithread indicou que 4 *threads* representam o limite físico (Ponto Doce) da FPU do processador testado; o uso de *threads* lógicas adicionais (Hyper-Threading) causou degradação por sobrecarga de troca de contexto (*Context Switch*).

5 Conclusão

O projeto provou que a otimização de *software* não se resume a ativar *flags* de compilador. Para atingir o estado da arte em HPC, o engenheiro deve alinhar o rigor matemático (Método Explícito via Multiplicadores de Lagrange) ao conhecimento da física dos transistores (Cache e Memória Contígua). A implementação em C conseguiu superar a velocidade nativa do Python, entregando o mesmo PCA com precisão e estabelecendo um *speedup* absoluto de mais de 1300 vezes em relação ao protótipo ingênuo original.

Referências

- [1] Jolliffe, I. T., & Cadima, J. (2016). *Principal component analysis: a review and recent developments*. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 374(2065), 20150202.
- [2] Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press.
- [3] OpenMP Architecture Review Board. (2021). *OpenMP Application Program Interface Version 5.2*.
- [4] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830.